

NAME: _____

CS3490 Midterm

For each problem, implement the functions satisfying the given specification.

You are free to define helper functions whenever you need to. You can also use built-in functions.

Don't copy the types of the functions.

Start writing the code at the left edge.

If necessary, you may abbreviate long function names to save space.

1. List Processing

1. Write a function `lookUp :: (Eq a) => a -> [(a,b)] -> b` where, given an element `x :: a`, and a list `env :: [(a,b)]` of key-value pairs, the function call `lookUp x env` outputs the value associated to key `x`, if it occurs in the list `env`.

The behavior of the function is undefined if the key does not occur, so it can output anything in that case. (Or give an error.)

```
> lookUp 2 [(0,True),(1,False),(2,True)] = True
> lookUp 1 [(0,True),(1,False),(2,True)] = False
```

2. `subset :: (Eq a) => [a] -> [a] -> Bool` takes two lists, `l1` and `l2`, and returns `True` if every element in `l1` is also present in `l2`.

```
> subset [1,2,3] [1,2] = False
> subset [1,2,2,3] [5,4,3,2,1] = True
> subset [] [1,2] = True
```

3. `findPali :: [String] -> Maybe String` takes a list of strings, and if the list contains a palindrome, outputs that string inside the `Just` constructor. Otherwise, it should output `Nothing`. (If there are multiple palindromes, you can return any one of them.)

```
findPali ["Here","is","a","list","of","strings"] = Just "a"
findPali ["This","is","the","cs3490","midterm"] = Nothing
```

2. Combinators and Lambda Calculus

Recall the following λ -terms:

$$I = \lambda x.x$$

$$K = \lambda xy.x$$

$$S = \lambda xyz.xz(yz)$$

$$W = \lambda xy.xyy$$

Simplify the following term until a normal form with no redexes is reached, or a cycle is detected. You can use the rules of combinatory logic to take shortcuts with lambda reductions.

1. WSIWI

2. $(\lambda x.x(\lambda y.yx))KSI$

3. Evaluate the following Haskell expression:

```
(\x y z -> y x (x z)) reverse map ["hey","you"]
```

3. Higher-order functions

Provide a *one-line implementation* for the following functions.

(If it's helpful, you may first write an implementation using pattern-matching and recursion. However, you will only receive credit for the one-line solution.)

1. `tagLength :: [String] -> [String]` takes a list of strings, adds a colon at the end of each, followed by the length of the string.
`> tagLength ["Hello","world","bye"] = ["Hello:5","world:5","bye:3"]`

2. `modPairs :: [(Integer,Integer)] -> [Integer]` takes a list of integers, and computes the remainders of the first number divided by the second number.
`> modPairs [(2,1),(5,3),(10,7)] = [0,2,3]`

3. `concatOdds :: [String] -> String` concatenates all strings in the given list which have **odd length**.
`> concatOdds ["Here","are","some","strings"] = "arestrings"`

4. Tree functions

Consider the following datatype of binary+unary trees in which data occurs *at the leaves* as well as *at binary nodes* but *not at unary nodes*.

```
data BTree a = BLeaf a | UNode (BTree a) | BNode a (BTree a) (BTree a)
```

```
myTree :: BTree Integer
myTree = BNode 3 (UNode (BLeaf 5))
           (BNode 4 (BLeaf 1) (UNode (BLeaf (-3))))
```

Implement the following functions.

1. `getLeaves :: BTree a -> [a]` returns the list containing all of the *leaves* of the tree.
`> getLeaves myTree = [5,1,-3]`

2. `squareTree :: BTree Integer -> BTree Integer` squares all the values in the tree.
`> squareTree myTree = BNode 9 (UNode (BLeaf 25))`
 `(BNode 16 (BLeaf 1) (UNode (BLeaf 9)))`

3. `allOdds :: BTree Integer -> Bool` returns `True` if *all* of the entries in the tree are odd.
`> allOdds myTree = False`

5. Propositional logic

Recall the Prop datatype from the last homework:

```
type Vars = String
data Prop = Var Vars | Const Bool | And Prop Prop | Or Prop Prop | Not Prop
```

Define the following functions.

1. `searchVars :: (a -> Bool) -> Prop -> Bool` takes a predicate on the variables, `p :: Vars -> Bool`, and a formula `f :: Prop` and returns `True` if *some* variable `x` occurs in the formula satisfying `p`, and returns `False` otherwise.

```
> searchVars (== "X") (Or (Var "X") (Var "Y")) = True
> searchVars (\x -> length x > 1) (Or (Var "X") (Var "Y")) = False
> searchVars (odd . length) (Or (Var "X") (Var "Y")) = True
```

2. `depth :: Prop -> Integer` returns the depth of the syntax tree of the formula `p`, i.e., the maximum number of binary or unary nodes on the path from the root to the leaf of the tree
`> depth (And (Not (Var "X")) (Or (Var "X") (Var "Z"))) = 2`

3. `subst :: Vars -> Prop -> Prop -> Prop` takes a variable `x`, a propositional formula `q`, and a formula `p`. It returns the formula `p` with *every* occurrence of the variable `x` replaced by `q`.

```
> subst "Y" (Const True) (Or (Var "X") (Var "Y")) = (Or (Var "X") (Const True))
> subst "Z" (Const True) (Or (Var "X") (Var "Y")) = (Or (Var "X") (Var "Y"))
> subst "Y" (Or (Var "Z") (Var "X")) (Or (Var "X") (Var "Y"))
    = (Or (Var "X") (Or (Var "Z") (Var "X")))
```

4. (**Extra credit 1.**) Recall the following classification of formulas in propositional logic.
- A formula is *satisfiable* if there *exists* a variable environment making the formula true.
 - A formula is a *contradiction* if *no* variable environment makes the formula true.
 - A formula is a *tautology* if *every* variable environment makes the formula true.
 - A formula is a *contingency* if some environments make it true, and some make it false: it is *neither* a contradiction *nor* a tautology.

Assume you have implemented a function `sat :: Prop -> Bool` which outputs whether a formula is *satisfiable*.

Use the `sat` function as a helper to implement a function `cont :: Prop -> Bool` which checks whether a formula is a *contingency*. Your solution should fit on one line.

(*Hint.* Think about what being a contingency means in terms of satisfiability.)

```
> cont (Or (Var "X") (Not (Var "X"))) = False
> cont (Or (Var "X") (Not (Var "Y"))) = True
```

5. (**Extra credit 2.**) Consider the following generic fold for propositions.

```
foldProp :: (Vars -> b) -> (Bool -> b) -> (b -> b -> b) -> (b -> b -> b) -> (b -> b)
          -> Prop -> b
foldProp v c a o n (Var x)      = v x
foldProp v c a o n (Const y)   = c y
foldProp v c a o n (And p1 p2) = a (foldProp v c a o n p1) (foldProp v c a o n p2)
foldProp v c a o n (Or p1 p2)  = o (foldProp v c a o n p1) (foldProp v c a o n p2)
foldProp v c a o n (Not p)     = n (foldProp v c a o n p)
```

Implement the `searchVars` function in one line using the above fold.